

Module 06: Learning — Implementing Parameter Learning

Course: Active Inference 101 | **Unit:** Implementation | **Audience:** First-semester undergraduates

Learning Objectives

1. Implement **Dirichlet parameter learning** for the A and B matrices.
2. Code the **learning rate decay** that naturally emerges from concentration parameters.
3. Build an agent that **learns from experience** and improves over time.

Key Concepts

1. Dirichlet Parameter Learning

```

import numpy as np

class LearningAgent:
    """Active Inference agent with Dirichlet parameter learning."""

    def __init__(self, A_prior, B_prior, C, D, gamma=1.0):
        # Store concentration parameters (Dirichlet priors)
        self.a = A_prior.copy() # shape: (num_obs, num_states)
        self.b = B_prior.copy() # shape: (num_states, num_states, num_ac
        self.C = C
        self.D = D.copy()
        self.gamma = gamma

        # Current point estimates
        self.A = self._dirichlet_mean(self.a, axis=0)
        self.B = self._dirichlet_mean_3d(self.b)

        self.qs = D.copy()
        self.learning_log = []

    @staticmethod
    def _dirichlet_mean(alpha, axis=0):
        """Compute mean of Dirichlet:  $E[\theta_i] = \alpha_i / \sum \alpha$ ."""
        return alpha / alpha.sum(axis=axis, keepdims=True)

    @staticmethod
    def _dirichlet_mean_3d(alpha):
        """Compute Dirichlet mean for 3D array (B matrix)."""
        result = np.zeros_like(alpha)
        for a in range(alpha.shape[2]):
            result[:, :, a] = alpha[:, :, a] / alpha[:, :, a].sum(axis=0, keepd

```

```

        return result

def update_A(self, observation, state_beliefs):
    """
    Update A matrix concentration parameters.

    a(o, s) += q(s) for the observed o
    """
    obs_vec = np.zeros(self.a.shape[0])
    obs_vec[observation] = 1.0

    # Outer product: update = P(o) ⊗ q(s)
    self.a += np.outer(obs_vec, state_beliefs)
    self.A = self._dirichlet_mean(self.a, axis=0)

def update_B(self, action, state_beliefs_prev, state_beliefs_curr):
    """
    Update B matrix concentration parameters.

    b(s', s, a) += q(s') × q(s) for the executed action
    """
    self.b[:, :, action] += np.outer(state_beliefs_curr, state_beliefs_prev)
    self.B = self._dirichlet_mean_3d(self.b)

def get_effective_learning_rate(self):
    """Compute the effective learning rate from concentration parameters"""
    total_a = self.a.sum()
    return 1.0 / total_a

def step(self, observation, action=None):
    """One step: infer, learn, act."""
    prev_qs = self.qs.copy()

```

```
# Infer states
self.qs = self.A[observation, :] * self.qs
self.qs /= self.qs.sum()

# Learn A matrix
self.update_A(observation, self.qs)

# Learn B matrix (if we have a previous action)
if action is not None:
    self.update_B(action, prev_qs, self.qs)

# Log learning rate
self.learning_log.append({
    'lr': self.get_effective_learning_rate(),
    'a_sum': self.a.sum(),
    'beliefs': self.qs.copy()
})

return self.qs
```

2. Tracking Learning Progress

```
def run_learning_experiment(agent, env, num_episodes=5, steps_per_episode=1000):
    """Run multiple episodes and track learning."""
    all_rewards = []

    for ep in range(num_episodes):
        env.reset()
        episode_reward = 0

        obs = np.random.choice(agent.A.shape[0], p=env.A[:, env.state])

        for t in range(steps_per_episode):
            agent.step(obs)
            action = agent.select_action()
            obs = env.step(action)

            # Track performance
            if obs == 0: # reward observation
                episode_reward += 1

        all_rewards.append(episode_reward)
        print(f"Episode {ep}: reward = {episode_reward}, "
              f"learning_rate = {agent.get_effective_learning_rate():.4f}")

    return all_rewards
```

Summary

Parameter learning updates concentration parameters of Dirichlet distributions governing A and B matrices. The effective learning rate decreases naturally with experience. Over episodes, the agent's model converges toward the true environment dynamics.

Further Reading

- Friston, K. J. et al. (2016). Active inference and learning. *Neuroscience & Biobehavioral Reviews*, 68, 862-879.